

<https://drive.google.com/file/d/1M5EriQ5tMaxJv97JHjDXy5cYFljE2m3y/view?pli=1>

K-Nearest Neighbors (KNN)

Instance Based Classifiers

Instance-based classifiers are machine learning methods that:

- Store training examples directly
- Do not build a fixed model beforehand
- Compare new data with already stored examples
- **Classify based on similarity**

👉 They are called **lazy learners** because they **delay learning until prediction time**.

Examples of Instance-Based Classifiers

1. Rote Learner

- Memorizes the entire training dataset
- Performs classification **ONLY** if:
 - **new record matches a stored example exactly**
- If the new record is **slightly different**: **✗ It cannot classify properly**

2. Nearest Neighbor

- Looks for the **closest training points**
- **Uses similarity/distance** to classify unknown data
- Instead of exact matching checks **“Which stored examples are most similar?”**

★ K-Nearest Neighbors (KNN)

- Supervised Learning
- This means:
 - algorithm learns from labeled examples
 - Then predicts labels for new unseen records
- Similar objects usually belong to the same class
- Nearby data points tend to have similar labels

🎯 What KNN Requires

📁 1. Set of Stored Records

- This is the training dataset.
- Contains:
 - Input features
 - Correct class labels

📏 2. Distance Metric

A way to calculate: 🖐️ How far two records are from each other

Common distance measures:

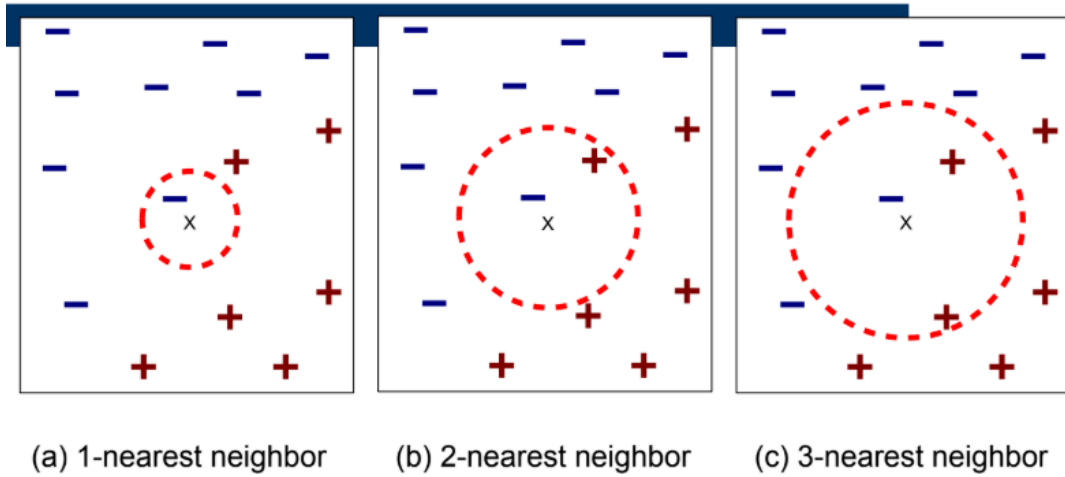
- Euclidean Distance
- Manhattan Distance

📊 3. Value of K

K = Number of nearest neighbors considered.

- $K = 1 \rightarrow$ Use nearest point only

- $K = 3 \rightarrow$ Use 3 closest neighbors



> K-nearest neighbors of a record x are data points that have the k smallest distance to x

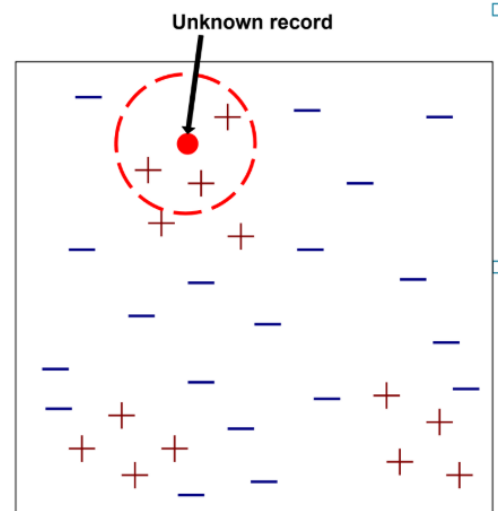


Unknown Record in KNN

Nearby points help determine its class.

- + may represent one class
- - may represent another class

The dashed circle shows nearby neighbors considered for classification.



Steps of KNN Classification

Step 1 Compute Distance

Calculate distance between:

- Unknown record
- Every training record

Step 2 Find K Nearest Neighbors

- Choose the **K closest points**.
- These are called: **K-Nearest Neighbors** [data points that have the **K smallest distances** to **x**]

Step 3 Majority Voting

- Look at **class labels of neighbors**.
 - The class **occurring most frequently becomes the predicted class**.
-

Understanding Different Values of K

♦ $K = 1$

- Only nearest point considered
- **Very sensitive to noise**
- Fast but unstable

♦ $K = 3$

- More reliable
 - Small voting system
- ♦ K = 5 or higher
- Smoother decision boundary
 - Less sensitive to noise
 - may ignore local patterns
-



Distance Metrics in KNN

Smaller distance = More similarity



Euclidean Distance

- Used for straight-line distance.

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$



Manhattan Distance

- Moves only horizontally and vertically.
- Absolute difference

$$d(p, q) = \sum_i |p_i - q_i|$$

Advantages of kNN

- Simple, intuitive, and easy to implement
- **Only one hyperparameter (k) to tune**
- No assumptions about data distribution
- Naturally **handles multiclass problems**
- **No training** phase

Disadvantages of kNN

- **Sensitive to the choice of k**, distance function, and noisy data
- **Lazy learner** – no precomputed model (all computation at test time)
- **No training cost**
- **very large computation and storage cost** for large datasets
- All **features treated equally** (noisy features degrade performance)



Training Data

ID	F1	F2	Class
T1	1	5	0
T2	0	8	0
T3	0	6	1
T4	1	2	1



Test Data

ID	F1	F2	Class
S1	1	3	?
S2	1	4	?
S3	0	3	?
S4	0	4	?

This example uses **Manhattan Distance**:

$$d(p, q) = \sum_i |p_i - q_i|$$

For 2 features:

$$d = |F1_1 - F1_2| + |F2_1 - F2_2|$$

Distance from Test Sample 1 to All Training Samples		Class
1	$ 1-1 + 3-5 = 0 + 2 = 2$	0
2	$ 1-0 + 3-8 = 1 + 5 = 6$	0
3	$ 1-0 + 3-6 = 1 + 3 = 4$	1
4	$ 1-1 + 3-2 = 0 + 1 = 1$	1

Nearest distance:

1

Nearest neighbor:

👉 T4

Class of T4:

1

Final Prediction

Test Sample	Predicted Class
(1,3)	1

Distance Table for Sample 2

Training Sample	Distance	Class
T1	1	0
T2	5	0
T3	3	1
T4	2	1



Distance Table for Sample 3

Training Sample	Distance	Class
T1	3	0
T2	5	0
T3	3	1
T4	2	1

ID	Actual	Predicted
1	0	1
2	0	0
3	1	1
4	1	0 or 1



Scaling Issues in Nearest Neighbor Classification

- KNN **classifies** data **based on distance between records**.
- The distance is calculated using all attributes (features).
- If **one attribute has much larger values** than others, it **can dominate** the distance calculation and **make the smaller attributes almost irrelevant**.
- As a result, the classification may become **biased**.
- Different attributes can have different ranges. ⇒ so we need scaling!!!!!!
 - height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 90lb to 300lb
 - income of a person may vary from \$10K to \$1M

Nearest Neighbor (NN) Classifier with Normalization

In the previous example, **F2 values (2, 5, 6, 8)** had a larger range than **F1 values (0, 1)**. To ensure both attributes contribute fairly to distance calculations, we **normalize the data to the range [0, 1]**.

Using **Min-Max Normalization**:

$$x' = \frac{x - MinV}{MaxV - MinV}$$

- x = original value
- $MinV$ = minimum value of the attribute
- $MaxV$ = maximum value of the attribute
- x' = normalized value

Step 1: Normalize F1

> not normalized:

F1	F2	Class
1	5	0
0	8	0
0	6	1
1	2	1

F1 values:

1, 0, 0, 1

Minimum:

$$\underline{MinV = 0}$$

Maximum:

$$\underline{MaxV = 1}$$

Applying normalization:

For F1 = 1

$$\frac{1 - 0}{1 - 0} = 1$$

For F1 = 0

$$\frac{0 - 0}{1 - 0} = 0$$

So F1 remains unchanged:



Step 2: Normalize F2

Original F2 values:

5, 8, 6, 2

Minimum:

$$\underline{MinV = 2}$$

Maximum:

$$\underline{MaxV = 8}$$

Range:

$$8 - 2 = 6$$

◆ Training Sample 1

F2 = 5

$$\frac{5 - 2}{8 - 2} = \frac{3}{6} = 0.5$$

◆ Training Sample 2

F2 = 8

$$\frac{8 - 2}{6} = 1$$

◆ Training Sample 3

F2 = 6

$$\frac{6 - 2}{6} = \frac{4}{6} = 0.667$$

Normalized Training Data

F1	F2	Class
1	0.5	0
0	1	0
0	0.667	1
1	0	1

Step 3: Normalize Test Data

1	3	?
1	4	?
0	3	?
0	4	?

becomes

1	0.167	?
1	0.334	?
0	0.167	?
0	0.334	?

Classification After Normalization

Using the normalized data and applying **1-Nearest Neighbor**, the slide gives:

ID	Actual	Predicted
1	0	1
2	0	0
3	1	1
4	1	1

Without normalization:

- Attributes with **larger ranges dominate**.
- **Distances** may be **misleading**.
- **Ties and incorrect classifications can occur**.

With normalization:

- All attributes are on the **same scale**.
- **Distances** become **fair**.
- Nearest neighbors are **selected more accurately**.
- **Classification** performance usually **improves**.

Steps in Normalized KNN

1. Find minimum and maximum values.
2. Normalize all attributes using Min-Max normalization.
3. Compute distances on normalized data.
4. Find nearest neighbor(s).
5. Assign class label using nearest neighbor or majority vote.

✓ **Normalization is strongly recommended before applying KNN because KNN is completely distance-based.**



Confusion Matrix

A **Confusion Matrix** is a table used to [evaluate the performance of a classification algorithm](#).

It compares:

- **Actual Class** (true value)
- **Predicted Class** (model output)

and [shows how many predictions were correct or incorrect](#).

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

✓ **TP: correct predictions that an instance is positive**

✓ **TN: correct predictions that an instance is negative**

✗ FP [Type-I Error]: **incorrect predictions** that an **instance is positive**

✗ FN [Type-II Error]: **incorrect predictions** that an **instance is negative**



Confusion Matrix from KNN Example

Confusion Matrix from the example of Lecture 2
(without Normalization)

ID	Actual	Predicted
1	1	1
2	0	0
3	1	1
4	1	0

	Negative	Positive
Negative	TN = 1	FP = 0
Positive	FN = 1	TP = 2



Accuracy (AC)

- **How many predictions** were **correct** out of all predictions.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

$$Accuracy = \frac{2 + 1}{2 + 1 + 0 + 1}$$

✓ **Accuracy = 75%**

Recall / True Positive Rate (TPR)

- Out of all actual positive cases, **how many were correctly identified?**

$$TPR = \frac{TP}{TP+FN}$$

$$TPR = \frac{2}{2+1} \quad \checkmark \text{ Recall (TPR) = 66.7\%}$$

False Positive Rate (FPR)

Out of all actual negatives: How many were **incorrectly classified as positive?**

$$FPR = \frac{FP}{FP+TN}$$

$$FPR = \frac{0}{0+1} \quad \checkmark \text{ FPR = 0\%}$$

True Negative Rate (TNR)

Out of all actual negative cases: **How many were correctly classified?**

$$TNR = \frac{TN}{TN+FP}$$

False Negative Rate (FNR)

Out of all actual positives: How many were **incorrectly classified as negative**?

$$FNR = \frac{FN}{FN+TP}$$

Precision (P)

Out of all predicted positive cases, **how many were actually positive**?

$$Precision = \frac{TP}{TP+FP}$$

F1-Score (F-Measure)

F1 Score **combines**:

- **Precision**
- **Recall**

into a single metric.

It is the **harmonic mean** of Precision and Recall.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

ROC Curve?

- ROC stands for: **Receiver Operating Characteristic Curve**
- graph **used to evaluate the performance of a binary classification model.**

Instead of checking performance at only one threshold, ROC shows how the classifier performs at **all possible classification thresholds.**

Why Do We Need ROC?

Many **classifiers output probabilities.**

Sample	Probability of Positive
A	0.95
B	0.80
C	0.45
D	0.20

Using threshold = 0.5:

- A → Positive
- B → Positive
- C → Negative
- D → Negative

Using threshold = 0.3:

- A → Positive
- B → Positive
- C → Positive
- D → Negative

Changing the threshold changes:

- TP
- TN
- FP
- FN

ROC shows the effect of all these threshold changes.

The ROC curve plots:

Axis	Metric
X-Axis	False Positive Rate (FPR)
Y-Axis	True Positive Rate (TPR)(Recall)

$$Recall = TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$



Alternative Representation

Sometimes performance is shown using: **Precision vs Recall**

Instead of: **TPR vs FPR**

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

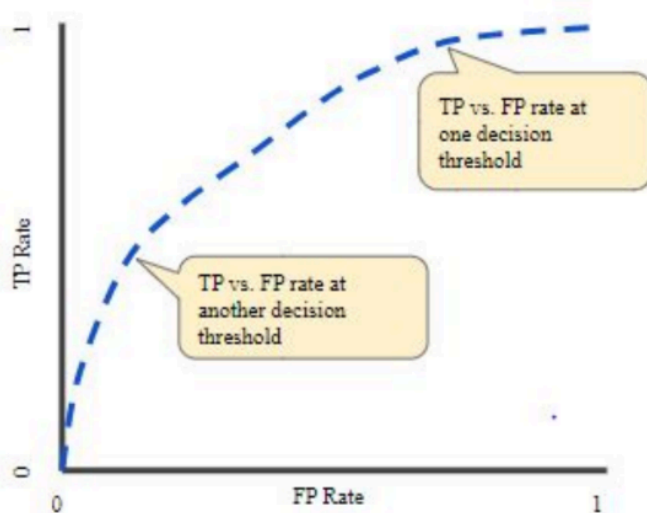


How ROC Curve is Generated

The **classifier** is **evaluated repeatedly** using **different thresholds**.

Threshold	TPR	FPR
0.9	0.20	0.01
0.7	0.50	0.05
0.5	0.75	0.15
0.3	0.90	0.30
0.1	1.00	1.00

- Each pair: (FPR, TPR) becomes a point on the graph.
- Connecting these points forms the **ROC Curve**.



🎯 Effect of Lowering Threshold

When **threshold decreases**: **More records are classified as positive**.

As a result: **✓ TP increases** BUT **✗ FP also increases**



Interpreting ROC Curve

Good Classifier

Curve **rises sharply toward the top-left corner.**

Characteristics:

- **High TPR**
- **Low FPR**

Ideal situation.

Average Classifier

- Curve **near diagonal line.**
- Performance similar to random guessing.

Poor Classifier

- Curve **close to bottom-right.**
- Produces many wrong predictions.

Ideal ROC Point

Best possible point: **(FPR=0, TPR=1)**

Meaning:

- **No false positives**
- **All positives detected**

Located at: **Top-Left Corner**



AUC (Area Under Curve)

- AUC stands for: **Area Under the ROC Curve**
- It measures the **overall quality of a classifier**.

Instead of comparing many ROC points, **AUC summarizes the entire ROC curve into one number.**

AUC	Meaning
1.0	Perfect Classifier
0.9 – 0.99	Excellent
0.8 – 0.9	Very Good
0.7 – 0.8	Good
0.6 – 0.7	Fair
0.5	Random Guessing
< 0.5	Worse than Random



Example from Slide

Three background subtraction techniques are compared:

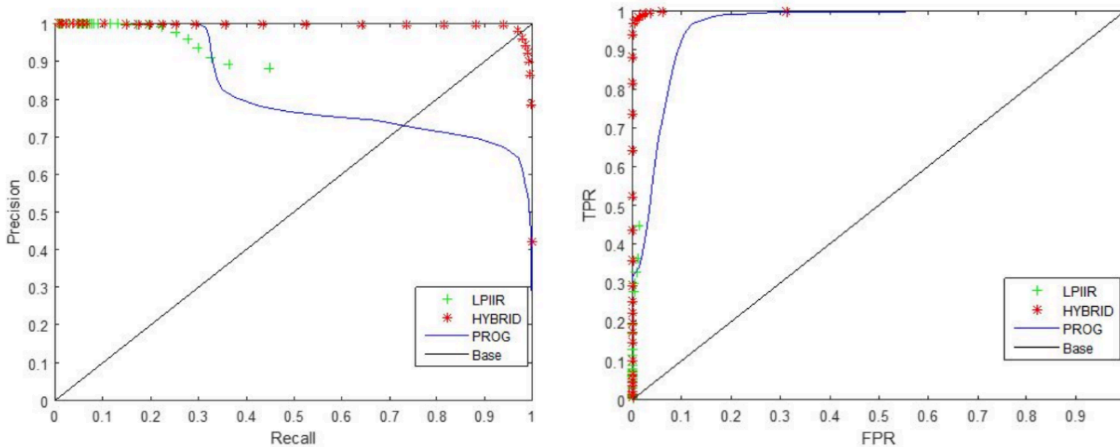
- LPIR
- Progressive Generation
- Hybrid

Observation:

✓ Hybrid technique has:

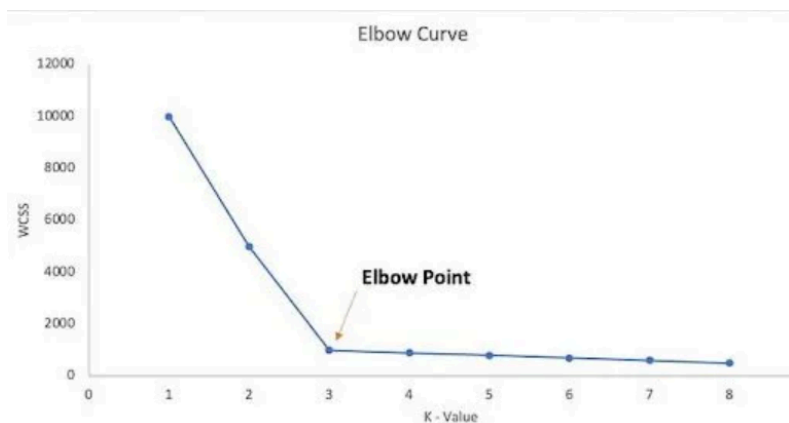
- Higher Precision, Higher Recall
- Higher TPR, Lower FPR

Therefore: ✓ **Hybrid method** performs best among the three.



Elbow Method (K-Means Clustering)

- The Elbow Method helps determine: **Optimal Number of Clusters (K)**
- The Elbow Method uses: **WCSS (Within-Cluster Sum of Squares)**
- WCSS measures: **How compact the clusters are.**
- **Lower WCSS** means: ✓ **Better clustering**



Axis	Value
X-Axis	Number of Clusters (K)
Y-Axis	WCSS

As **K increases**:

- **WCSS decreases**
 - Clusters become more compact
-
- Initially: **WCSS drops sharply**
 - After some point: **Improvement becomes very small**
 - The graph forms an "elbow".

The optimal K is: **The point where the graph bends like an elbow.**

At this point:

- **Most clustering improvement has already been achieved**
- **Adding more clusters gives little benefit**

K	WCSS
1	10000
2	5000
3	1200
4	1000
5	900

Huge improvement until K = 3.

After that, **WCSS decreases only slightly.**

✓ **Optimal K = 3**

ROC Curve

- Used for classification evaluation
- **X-axis = FPR**
- **Y-axis = TPR**
- AUC measures overall performance

Elbow Method

- Used in K-Means clustering
 - Finds optimal number of clusters
 - **X-axis = K**
 - **Y-axis = WCSS**
 - **Elbow point = Best K value**
-